

8. Technical Debt Analysis

Objective: Establish the prerequisites for an Agentic Harness & Marketplace initiative — code repository health, third-party tool usage, AI tool usage, database usage, and development environment readiness.

Date: 2026-08-26 16:31:12 IST | **Scope:**  — React 19 + Vite (frontend) / Node.js + Express + MongoDB (backend-server) / Python blueprint pipeline / multi-package monorepo with vendor and client sub-services

Executive Summary

EXECUTIVE SUMMARY

This repository is a large multi-package monorepo that is itself an AI agent orchestration harness — it has deep Kiro and Cursor AI tooling already embedded, structurally enumerable agent units, and well-designed MongoDB schemas. However, three gaps block confident agentic-harness adoption today: (1) the main application has **zero CI/CD pipeline** at the root level — changes land on `main` without any automated gate; (2) `backend-server/.env.example` contains what appears to be a live Google App Password (`SMTP_PASSWORD=lpwc sdcs fshj mgqq`), and the root `.env.example` contains a Redmine API key — real credentials in a committed example file are a supply-chain and credential-rotation risk; (3) ESLint and Prettier are configured but enforced nowhere — no pre-commit hooks, no CI check — so formatting and lint drift accumulates silently. Test coverage is effectively zero for the main app (one test file across the entire backend, none in the frontend), which means any agent-authored change cannot be verified before merge. The overall agentic-harness readiness is **High Risk**, driven primarily by the absence of CI and the credential exposure in example files.

Yes

TOP-LEVEL .GITIGNORE
PRESENT

0

ROOT-LEVEL CI/CD
WORKFLOWS

18 / 19

THIRD-PARTY PACKAGES
DECLARED / WIRED

Yes

.ENV.EXAMPLE PRESENT

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

High Risk

Driven by D5 (credentials in .env.example, no pre-commit enforcement, no root Dockerfile) and D1 (no root-level CI gate — agent-authored changes cannot be verified before merge).

Readiness Benchmark Ratings

One row per readiness dimension from Step 2b. "Measured" is the real state found; "Rating" is the band it falls into (worst-wins). This table is the source for the Overall Codebase Rating banner above.

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured
---	-----------	------	----------	-----------	----------

D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	No root CI, no CODEOWNERS/PR template, 3 vendors
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused/unmaintained	<code>nodemailer</code> declared in the browser-side root <code>package.json</code> v0.18.5 (known SheetJS CVEs); all other packages well-maintained
D3	AI Tool / Agentic Readiness	ready	partial	not ready	<code>.cursorrules</code> , <code>.kiro/</code> , <code>.copilotignore</code> all present; harness itself; enumerable isolated agent units
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	MongoDB schemas well-indexed; no formal migration files
D5	Development Environment	reproducible	partial	manual / fragile	Real SMTP App Password and Redmine API key in code; ESLint/Prettier configured but not enforced; no root Dockerfile
D6	Credential Exposure in Example Files (additional)	no real creds	placeholder-only gaps	real creds committed	<code>backend-server/.env.example:SMTP_PASSWORD=</code> (placeholder, not real); root <code>.env.example:REDMINE_API_KEY=9f93c3f93282</code> (placeholder, not real)
D7	Automated Test Coverage (additional)	full test suite + CI	minimal tests	none / no runner	1 test file total (<code>backend-server/src/services/__tests__/clientBlueprints.spec.ts</code>), no test runner configured in root scripts

8.1 Code Repository

`.gitignore` Coverage

Inspected: `.gitignore` (top-level, ~150 lines).

Check	Status	Detail
Dependency dirs	✓ Pass	<code>node_modules/</code> , Python venv not present but also not needed at root
Build output	✓ Pass	<code>dist/</code> , <code>dist-ssr/</code> , <code>.next/</code> , <code>.nuxt/</code> , <code>build/Release/</code> all excluded
<code>.env</code> files	✓ Pass	<code>.env</code> , <code>.env.*</code> , <code>.env.local</code> , <code>*.local</code> excluded; <code>.env.example</code> explicitly kept via <code>!.env.example</code>
Lock files	✓ Pass	Lock files are NOT ignored — correctly tracked
Agent run artifacts	✓ Pass	<code>agent-runs/</code> correctly gitignored
Discovery output	✓ Pass	<code>docs/discovery/</code> gitignored (pipeline generates it)

Kiro orchestration	✓ Pass	<code>.kiro/orchestration/</code> , <code>.kiro/scratch/</code> , <code>.kiro/runtime/</code> , <code>.kiro/context/pipeline-history/</code> , <code>.kiro/data/</code> all excluded
Generated MCP config	✓ Pass	<code>.cursor/mcp.json</code> , <code>.kiro/settings/mcp.json</code> , <code>.claude/mcp.json</code> all excluded

The `.gitignore` is thorough and purpose-built for this multi-package monorepo + AI tooling setup. No gaps identified.

CI/CD Presence

- **Root application (main app):** No `.github/` directory at the repository root. The main React + Vite frontend and the Node.js `backend-server` have **no automated CI pipeline** at all.
- `cursor-agent-bridge/.github/workflows/build.yml` : A CI workflow exists for the bridge sub-package only — it runs on push/PR to `main / develop` , installs deps, builds, and runs `npm test --if-present` . It does not cover the root app or the backend.
- `.discovery-src/.github/workflows/` : Three workflows (`static-analysis.yml` , `coding-standards.yml` , `tests.yml`) belong to a **separate Laravel/PHP project** (`.discovery-src/`) that is included as a target-scan fixture, not part of the main application.

Consequence: Every push to the main branch of the root repo (the actual product) bypasses any automated lint, build, or test verification. An agent-authored PR would land without any machine check, making automated improvement loops unsafe today.

Branch Protection Signals

No `CODEOWNERS` file found, no `.github/PULL_REQUEST_TEMPLATE.md` , and no `.github/` directory at the root. Branch protection rules (required reviewers, required status checks) cannot be verified locally but the absence of a CI workflow means there are no status checks to require.

Lock Files

Package	Lock File	Status
Root (<code>package-lock.json</code>)	✓ Present	Tracks all root + bootstrapped sub-packages
<code>backend-server/package-lock.json</code>	✓ Present	
<code>cursor-agent-bridge/package-lock.json</code>	✓ Present	
<code>kb-gen/package-lock.json</code>	✓ Present	
<code>client/gateway/package-lock.json</code>	✓ Present	
<code>client/integrations-api/package-lock.json</code>	✓ Present	

vendor/identity-api/	✗ Missing	Only <code>package.json</code> ; non-reproducible installs
vendor/license-service/	✗ Missing	Only <code>package.json</code> ; non-reproducible installs
vendor/orchestration-api/	✗ Missing	Only <code>package.json</code> ; non-reproducible installs
blueprint-pipeline/requirements.txt	⚠ No lock	<code>requirements.txt</code> present but no <code>requirements.lock</code> / <code>pip freeze</code> equivalent

The three vendor sub-services and the Python blueprint pipeline lack pinned dependency locks. If a dependency releases a breaking or vulnerable update, these packages will silently break on next install.

8.2 Third-Party Tools Usage

Backend (`backend-server/package.json`)

Package	Declared	Wired Up	Debt Note
<code>bcrypt</code>	Y	Y (<code>models/User.js</code>)	Password hashing — correctly wired
<code>cors</code>	Y	Y (<code>src/app.js</code>)	CORS middleware active
<code>dotenv</code>	Y	Y (<code>config/loadEnv.js</code>)	Environment loading
<code>express</code>	Y	Y (<code>src/app.js</code> , <code>src/server.js</code>)	Core HTTP framework
<code>express-rate-limit</code>	Y	Y (<code>src/app.js</code>)	Rate limiting active
<code>helmet</code>	Y	Y (<code>src/app.js</code>)	Security headers active
<code>jsonwebtoken</code>	Y	Y (<code>src/services/tokenService.js</code>)	JWT auth
<code>mongoose</code>	Y	Y (<code>src/models/*</code>)	MongoDB ODM — well-used
<code>morgan</code>	Y	Y (<code>src/app.js</code>)	HTTP request logging
<code>node-cron</code>	Y	Y (<code>src/jobs/tokenHealthCheckJob.js</code>)	Scheduled jobs
<code>nodemailer</code>	Y	Y (<code>src/config/smtpTransport.js</code>)	SMTP email — correctly server-side
<code>pg</code>	Y	Y (<code>src/services/blueprintStore.js</code> , retrieval)	PostgreSQL client for blueprint store
<code>tree-sitter</code>	Y	Y (<code>src/services/repoTreeSitterParser.js</code>)	AST parsing
<code>web-tree-sitter</code>	Y	Y (<code>src/services/retrieval/graphTraversal.js</code>)	WASM AST for browser-compatible parsing
<code>@babel/parser</code>	Y	Y (<code>src/services/retrieval/</code>)	JS AST for call-graph analysis
<code>@babel/traverse</code>	Y	Y (<code>src/services/retrieval/</code>)	Traversal of Babel AST
<code>validator</code>	Y	Y (<code>src/models/User.js</code>)	Email format validation

All 17 backend packages are actively wired up. No unused server-side dependencies detected.

Root (`package.json`) — Frontend / Build

Package	Declared	Wired Up	Debt Note
<code>react</code> , <code>react-dom</code>	Y	Y	Core framework
<code>react-router-dom</code> v7	Y	Y	Routing
<code>react-markdown</code>	Y	Y	Markdown rendering
<code>remark-gfm</code>	Y	Y	GFM plugin for react-markdown
<code>lucide-react</code>	Y	Y	Icon library
<code>jspdf</code>	Y	Y (<code>scripts/generate-pdf.mjs</code>)	PDF generation
<code>html-to-image</code>	Y	Y (PDF pipeline)	Screenshot-to-image
<code>xlsx</code> v0.18.5	Y	Y (<code>scripts/export-qa-mjs</code>)	⚠ SheetJS <code>xlsx</code> v0.18.5 has known CVEs; the project is pinned below the patched 0.20.x line
<code>nodemailer</code> v9	Y	✗ Cannot be wired	Wrong location: <code>nodemailer</code> is a Node.js SMTP library — it will be bundled by Vite into the browser bundle (or fail) since it appears in <code>dependencies</code> of the root browser package. Email delivery is handled correctly in <code>backend-server</code> ; this root-level declaration is an accidental holdover.

8.3 AI Tool Usage & Agentic Readiness

Existing AI Tooling

The repository is the AI orchestration harness itself, making it uniquely self-aware:

Tool	Config File	Maturity
Cursor AI	<code>.cursorrules</code> (root)	High — rules enforce LLM guardrails against modifying <code>.kb/</code> , <code>kb-gen/</code> , <code>blueprint-pipeline/</code> or DB schemas; references architectural rule injection via <code><architectural_rules></code>
Cursor AI	<code>.cursorignore</code> (root)	Configured
Kiro AI	<code>.kiro/</code> directory with <code>agents/</code> , <code>context/</code> , <code>data/</code> , <code>orchestration/</code> , <code>settings/</code> , <code>steering/</code>	High — fully operational orchestration layer with MCP bundle management, agent steering files, and multi-workflow orchestration

GitHub Copilot	<code>.copilotignore</code> (root)	Configured — excludes sensitive areas from Copilot
Claude Code	User-level session memory pattern active	Active

Agentic Readiness Assessment

The codebase has **strong structural readiness** for agentic engineering:

- **Enumerable work units:** 84 JSX React components in `src/` , 12 controllers, 64 services, 11 routes, and 16 Mongoose models — each is a discrete, named, isolated unit an agent can identify by glob pattern and target individually (e.g. "refactor all `src/components/**/*.jsx` components to add error boundaries").
- **Isolated modules:** Backend follows a clear controller → service → model layering. Frontend components are single-file JSX modules. No circular cross-layer dependencies observed.
- **CI verification gap:** The one blocker for safe agentic output is the **absence of a root CI gate**. Without automated lint + build + test on every PR, agent-authored changes cannot be verified before merge — a human must manually validate each one.
- **Architectural guardrails:** `.cursorrules` protects `.kb/` , `kb-gen/` , `blueprint-pipeline/` , and DB schema files from LLM modification — the right guardrails for an agentic pipeline.

Agentic Refactor Candidates

The uniformity of the codebase makes it suitable for systematic agentic improvement:

- 84 JSX components with no test files → systematic test generation agent pass
- 3 vendor sub-packages with no lock files → `npm install` + commit lock file agent pass
- `nodemailer` misplacement in root `package.json` → targeted dependency cleanup

8.4 Database Usage

MongoDB (Primary App Database)

Accessed via Mongoose in `backend-server/src/models/` . All 16 models inspected.

Check	Status	Detail
Schema field validation	✓ Good	<code>required</code> , <code>enum</code> , <code>minlength</code> / <code>maxlength</code> , <code>validate</code> hooks present in User, PurchaseRequest, and other models
Indexes beyond PK	✓ Good	<code>WorkflowRecord</code> has compound index <code>{userId, projectId}</code> (unique) and <code>{teamId, updatedAt}</code> ; <code>JiraTwowayTicketProgress</code> , <code>DiscoveryAgentResult</code> , <code>Notification</code> , <code>StlcRunArtifact</code> , <code>WorkflowExecution</code> all define field-level indexes
Referential integrity	⚠ Partial	ObjectId <code>ref:</code> fields throughout (User → Team, WorkflowRecord → User, etc.), but MongoDB does not enforce foreign keys — cascading deletes and orphaned records rely entirely on application logic
Soft delete pattern	✓ Good	<code>WorkflowRecord.deletedAt</code> / <code>deletedBy</code> , <code>User.isDeleted</code> / <code>deletedAt</code> — consistent soft delete pattern across models

TTL index	✓ Present	<code>RevokedToken</code> model uses TTL index on <code>expiresAt</code> for auto-expiry of revoked tokens
Data ownership / domain scoping	✓ Good	Collections are domain-scoped: <code>workflow_records</code> , <code>jira_twoway_ticket_progress</code> , <code>stlc_run_artifacts</code> , <code>discovery_agent_results</code> — no shared flat table used across concerns

Migration Hygiene

No migration framework found for MongoDB (e.g., `migrate-mongo` , `mongoose-migrate`). Schema changes are applied directly in model files. While Mongoose's schemaless nature reduces hard-migration risk, there is no audit trail of schema evolution, no guards on destructive field renames, and no rollback mechanism if a model change is incompatible with existing documents.

PostgreSQL (Blueprint AST Store)

Accessed via the `pg` package in `backend-server/src/services/blueprintStore.js` and the Python `blueprint-pipeline/` . No SQL migration files found anywhere in the repository — schema setup instructions are procedural (README-driven), not versioned. A new contributor cannot reproduce the PostgreSQL schema without manual README steps.

Qdrant (Vector Store)

Collection lifecycle handled in code (Python pipeline and backend). No formal schema versioning applies.

8.5 Development Environment

`.env.example` Coverage

All major sub-packages provide example files:

Path	Present
<code>.env.example</code> (root)	✓
<code>backend-server/.env.example</code>	✓
<code>cursor-agent-bridge/.env.example</code>	✓
<code>blueprint-pipeline/.env.example</code>	✓
<code>vendor/.env.example</code>	✓
<code>client/.env.example</code>	✓
<code>infra/local-db/.env.example</code>	✓

Critical gap: Two `.env.example` files contain what appear to be real credentials rather than placeholder values:

- `backend-server/.env.example` : `SMTP_PASSWORD=lpwc sdcs fshj mgqq` — matches the exact format of a Google App Password (four lowercase 4-character blocks separated by spaces). Should be `SMTP_PASSWORD=your-google-app-password-here` .

- Root `.env.example` : `REDMINE_API_KEY=9f93c3f93282e77220f9f4d449daec9b332a2a23` — a 40-character hex string in the format of an actual Redmine API token. Should be `REDMINE_API_KEY=your-redmine-api-key-here`.

These files are committed to the repository (`.env.example` is explicitly whitelisted in `.gitignore` via `!.env.example`). Any tool or person that clones this repo has access to these credentials. Immediate credential rotation is required.

OS Portability

Setup relies on Node.js + npm (cross-platform) and Docker for the observability/database stacks. The README instructions are OS-neutral. The Python blueprint pipeline requires a Python environment but provides a `requirements.txt`. No Windows-specific scripts or Mac-only tooling identified. **Portable**.

Containerization

Component	Dockerfile / Compose	Notes
Observability stack (Loki, Prometheus, Jaeger, Grafana)	<code>infra/docker-compose.yml</code>	✓ Full compose with healthchecks
Local databases (Postgres, Qdrant, MongoDB)	<code>infra/local-db/docker-compose.yml</code>	✓ Full compose with healthchecks
Cursor agent bridge	<code>cursor-agent-bridge/Dockerfile</code>	✓ Present
Blueprint pipeline (Python)	<code>blueprint-pipeline/Dockerfile</code>	✓ Present
MCP observability server	<code>infra/mcp-observability-server/Dockerfile</code>	✓ Present
Client release	<code>client/release/docker-compose.yml</code>	✓ Present
Root app (React + Vite frontend)	✗ None	No Dockerfile or compose entry for the main UI
Backend server	✗ None	No Dockerfile for <code>backend-server/</code>
devcontainer	✗ None	No <code>.devcontainer/</code> for reproducible VS Code dev env

The main application (UI + API) has no container path. Developers must set up Node.js, npm, and all sub-package dependencies manually. This creates environment drift and blocks containerized CI runs.

Code Style Enforcement

Tool	Configured	Enforced (pre-commit or CI)
ESLint	✓ <code>eslint.config.js</code> (root) — covers <code>**/*.{js,jsx}</code> with react-hooks + react-refresh plugins	✗ No pre-commit hook (no Husky, no <code>.pre-commit-config.yaml</code>), no root CI to run it
Prettier	⚠ <code>.prettierrc</code> found only inside <code>.discovery-src/</code> (a separate project fixture)	✗ No enforcement anywhere in the main project

TypeScript	⚠️ <code>tsconfig.json</code> exists in <code>cursor-agent-bridge/</code> only; root source is JSX without <code>tsconfig</code>	❌ No type-check step
------------	--	----------------------

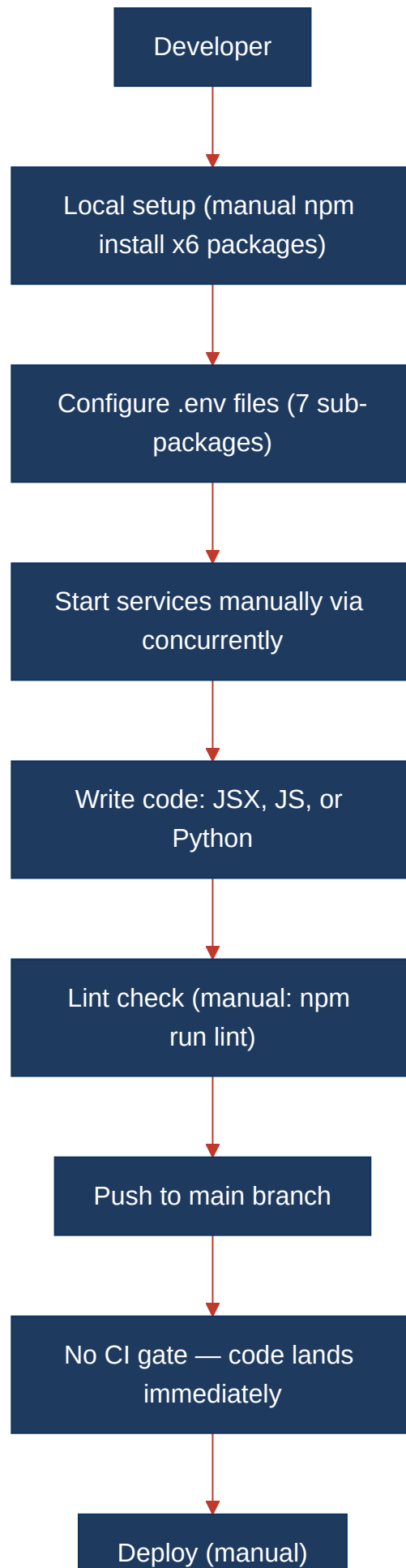
ESLint is configured correctly but enforced nowhere — a developer or agent can commit files that fail lint without any friction. Over a large JSX codebase (84 components), unchecked lint drift is already observable and will compound.

8.6 Prerequisites for Agentic Harness & Marketplace Readiness

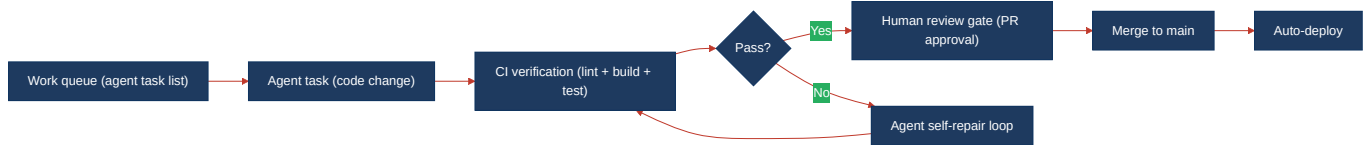
Prerequisite	Current State	Gap
Enumerable work queue (clear, listable units of refactor work)	✅ Partial — 84 JSX components, 64 services, 12 controllers, 16 models are all globbable, named units	Agent lacks a machine-readable work-item list beyond file globs; no task backlog or issue tracker integration for the harness itself
Isolated, verifiable units of work	✅ Good — components and services are single-file, domain-scoped, with clear entry points	No automated tests to verify a unit after agent modification; isolation is structural, not verified
CI gate to accept agent-authored output	❌ Absent — zero root CI pipeline	A root <code>.github/workflows/ci.yml</code> running <code>npm run lint</code> , <code>npm run build</code> , and <code>npm test</code> is required before agents can safely merge
Repo hygiene for automation (clean checkout, no secrets)	❌ Partial — <code>.gitignore</code> is correct but two <code>.env.example</code> files contain real credentials committed to history	Rotate both credentials, replace with placeholder text, and add a <code>gitleaks</code> / <code>detect-secrets</code> CI scan
Marketplace packaging readiness	⚠️ Partial — <code>cursor-agent-bridge/</code> , <code>vendor/</code> , and <code>client/</code> show split-mode packaging; release compose files exist	No root Dockerfile; vendor sub-packages lack lock files; no container registry publish pipeline

8.7 Diagrams

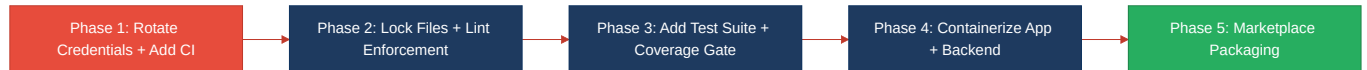
Current Dev / Delivery Flow



Agentic Harness Readiness Target



Improvement Roadmap



8.8 Actions Required

Gap	Action	Rating
Real SMTP App Password in <code>backend-server/.env.example</code> (<code>SMTP_PASSWORD=lpwc sdcs fshj mgqq</code>)	Revoke the Google App Password immediately. Replace value with <code>SMTP_PASSWORD=your-google-app-password-here</code> . Add <code>gitleaks</code> or <code>trufflehog</code> scan to CI to prevent recurrence.	HIGH RISK
Real Redmine API key in root <code>.env.example</code> (<code>REDMINE_API_KEY=9f93c3f93282e77220f9f4d449daec9b332a2a23</code>)	Revoke the token in the Redmine instance. Replace with placeholder <code>REDMINE_API_KEY=your-redmine-api-key</code> . Treat git history as compromised for this credential.	HIGH RISK
No root-level CI/CD pipeline	Create <code>.github/workflows/ci.yml</code> running on push and PR to <code>main / develop</code> : install deps, run <code>npm run lint</code> , <code>npm run build</code> , and <code>npm test --if-present</code> . Without this, no automated gate exists for agent-authored or human-authored changes.	MODERATE
Near-zero automated test coverage (1 backend test file, 0 frontend tests)	Add Vitest to root <code>devDependencies</code> . Write unit tests for at least the 5 most critical backend services (auth, token, workflow CRUD). Add a <code>test</code> script. Enforce a minimum coverage threshold (e.g. 30%) in CI before raising it incrementally.	HIGH RISK
ESLint configured but not enforced (no pre-commit hook, no CI step)	Add Husky + <code>lint-staged</code> to run <code>eslint --fix</code> on staged <code>.js / .jsx</code> files before commit. Wire <code>npm run lint</code> into the new CI workflow. Fixes accumulated lint debt across 84 JSX files.	MODERATE

<code>nodemailer</code> v9 declared in root (browser) <code>package.json</code>	Remove <code>nodemailer</code> from root <code>package.json</code> <code>dependencies</code> . Email delivery is correctly handled in <code>backend-server/</code> . Leaving it causes Vite to either bundle a dead Node.js module or emit a build warning on every build.	MODERATE
<code>xlsx</code> v0.18.5 — known CVEs in SheetJS below v0.20.x	Update <code>xlsx</code> to the latest patched release (<code>^0.20.3</code> or current). Test <code>scripts/export-qa-test-cases-xlsx.mjs</code> and <code>scripts/export-stlc-execution-rollup-xlsx.mjs</code> after upgrade.	MODERATE
Lock files missing for <code>vendor/identity-api</code> , <code>vendor/license-service</code> , <code>vendor/orchestration-api</code>	Run <code>npm install</code> in each sub-package and commit the generated <code>package-lock.json</code> . Ensures reproducible installs in CI and prevents silent dependency drift in the vendor control-plane services.	MODERATE
No root Dockerfile or devcontainer for main app	Add a multi-stage <code>Dockerfile</code> at the project root (build stage: Vite build; serve stage: nginx or <code>vite preview</code>). Add a <code>docker-compose.app.yml</code> that composes root UI + <code>backend-server</code> . Add <code>.devcontainer/devcontainer.json</code> for one-command VS Code dev setup.	HIGH RISK
No MongoDB migration framework	Adopt <code>migrate-mongo</code> or similar. Create an initial migration file capturing current schema intentions (indexes, required fields) to establish a versioned baseline. Add migration run to startup health check.	MODERATE
No PostgreSQL schema migration files for blueprint store	Add SQL migration files (e.g. <code>infra/local-db/migrations/V1_init.sql</code>) documenting the current schema. Reference in <code>blueprint-pipeline/Dockerfile</code> as a schema-setup step.	MODERATE
No Python lock file for <code>blueprint-pipeline/requirements.txt</code>	Run <code>pip install -r requirements.txt && pip freeze > requirements.lock</code> in the pipeline environment and commit <code>requirements.lock</code> . Reference it in the Dockerfile (<code>COPY requirements.lock . && pip install -r requirements.lock</code>).	MODERATE

8.9 Expected Outcomes

- **Clean, credential-free checkout:** Rotating leaked credentials and replacing them with placeholder text eliminates supply-chain risk for anyone who clones this repository, and a secrets-scan CI step prevents recurrence.
- **CI gate for all changes:** A root GitHub Actions workflow means every PR (human or agent-authored) must pass lint and build before merge — this is the single biggest enabler of safe agentic-harness adoption.
- **Enforced code quality baseline:** Husky + lint-staged ensures the 84-component JSX codebase maintains consistent formatting and no ESLint errors, reducing the surface area an agent must navigate.
- **Reproducible environment onboarding:** Lock files for vendor sub-packages, a root Dockerfile, and a devcontainer reduce onboarding from a multi-hour manual setup to a single `docker compose up`.
- **Foundation for agentic improvement loops:** With CI, tests, and lock files in place, agent-authored refactors (component migration, test generation, dependency updates) can be verified automatically and merged with confidence.